

Livello del Sistema Operativo

Livello del Sistema Operativo

Le risorse messe a disposizione dall'architettura del calcolatore vengono gestite da una componente software: il sistema operativo.

Possiamo vedere il sistema operativo come l'interfaccia fra il calcolatore ed i programmi applicativi (programmi utente: i sw installati sulla nostra macchina, ma anche i nostri programmi che eseguiamo).

Sistema Operativo e programmi applicativi

Il sistema operativo è un componente software trasversale, responsabile dell'esecuzione dei programmi applicativi, e che offre funzionalità necessarie alla loro esecuzione (funzionalità che non sono parte del programma stesso).

Tra i compiti principali del sistema operativo:

- Gestione della memoria
- Gestione dei processi (programmi in esecuzione)
- Gestione delle periferiche di I/O
- ...

Sistema Operativo e programmi applicativi

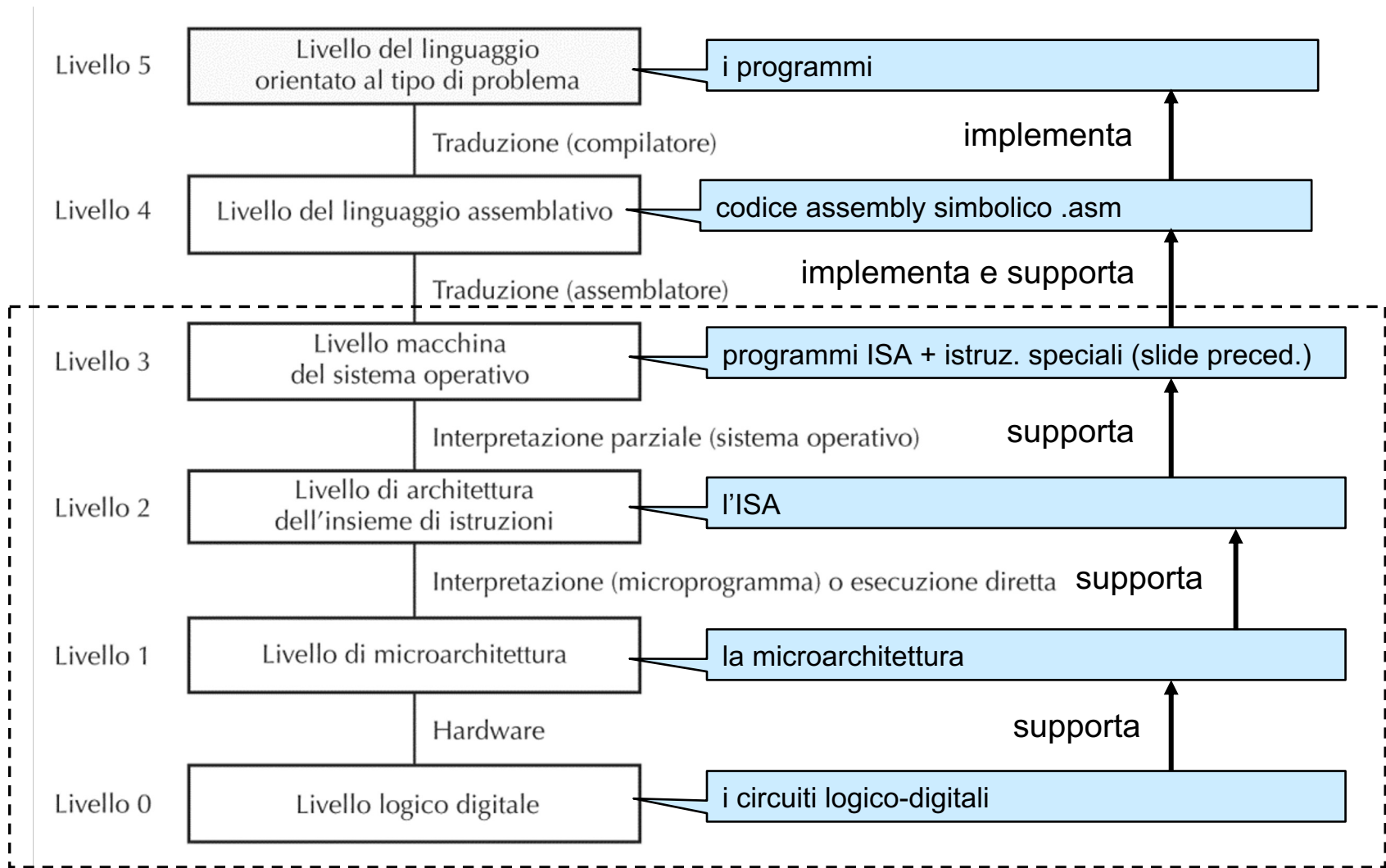


Figura 1.2 Computer a sei livelli. Sotto ciascun livello è indicato il metodo di supporto (oltre al nome del programma corrispondente).

Struttura di un tipico sistema operativo

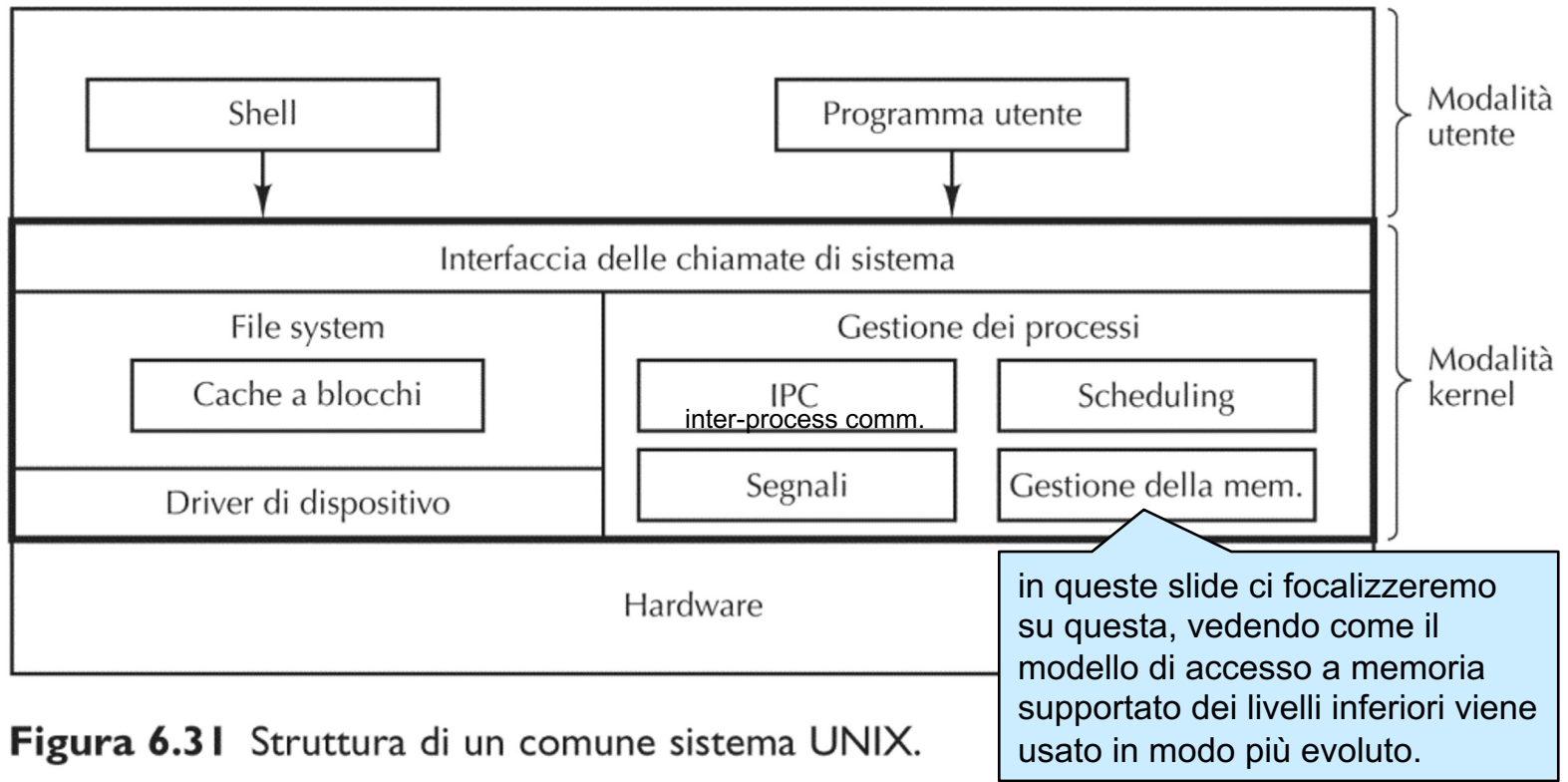


Figura 6.31 Struttura di un comune sistema UNIX.

Shell indica una specifica modalità per interagire con il sistema operativo (messa a disposizione dal cosiddetto terminale)

Shell e programmi applicativi interagiscono con il sistema operativo tramite “chiamate di sistema” (ad esempio, operazioni di accesso ai file, esecuzione di un programma, sospensione di un programma, ...)

Memoria virtuale: paginazione

Memoria virtuale: motivazione

Negli anni '60 i calcolatori avevano una memoria estremamente limitata. Pensiamo per esempio ad un computer a 16 bit con una memoria di 4K locazioni/indirizzi (non 4KB) contenenti parole da 16 bit e indirizzi a 16 bit.

Ovviamente, vorremmo avere una memoria superiore a 4K locazioni per salvare i programmi in esecuzione ed i loro dati e per utilizzarla come memoria di lavoro.

Notare:

Gli **indirizzi fisici** di memoria vanno quindi da 0 e 4095.

Ma $2^{16}=65536$, quindi gli indirizzi da 4096 a 65535 non vengono usati (piccola nota: una situazione simile si verifica nel computer Hack).

Memoria virtuale

L'idea è di sfruttare appieno la "capacità di indirizzamento" dei 16 bit, assumendo una memoria virtuale (non fisica!) da 64K locazioni, quindi usando tutti gli indirizzi da 0 a 65535.

Tali indirizzi vengono detti **indirizzi virtuali**.

Come è possibile assumere di avere a disposizione una memoria virtuale da 64K usando una memoria fisica (un chip) da 4K locazioni?

Idea: in ogni istante, al massimo 4K dati vengono posti in memoria primaria, usando per il resto **la memoria secondaria**. La difficoltà diventa scambiare continuamente il contenuto della memoria primaria, a seconda della necessità.

Tutto ciò in maniera trasparente al resto del sistema ed ai programmi: per quanto riguarda l'accesso a memoria, questa appare davvero da 64K locazioni.

Memoria virtuale: pro e contra

Pro: ovviamente, abbiamo "a disposizione" più memoria di lavoro senza dover aumentare la memoria fisica (non pensiamo solo al costo e dimensioni, ma anche alla complessità della microarchitettura).

Contra: la memoria secondaria è meno performante della memoria primaria, quindi stiamo creando un collo di bottiglia.

(le memorie RAM sono più veloci degli HDD sia per differenze tecnologiche dei componenti usati, sia per l'interfaccia di accesso, per esempio parallela vs seriale: possiamo avere più linee dati verso la RAM).

Aggiungere più memoria (RAM) fisica migliora le prestazioni.

Due problemi da risolvere

Primo problema: bisogna “**mappare**” (o "risolvere") gli indirizzi virtuali a cui si accede negli indirizzi fisici dove il dato è effettivamente salvato (assumendo che sia in memoria al momento).

Secondo problema: bisogna gestire la sostituzione continua dei dati richiesti dai programmi copiandoli davvero in memoria primaria, tenendo il resto in memoria secondaria.

L'approccio per risolvere entrambi è basato sulla **paginazione**.

Paginazione

- la memoria virtuale viene divisa in **pagine**, tutte di uguale dimensione (una potenza di due, ovviamente).
- Inoltre, la memoria primaria (fisica) viene suddivisa in parti di uguale dimensione chiamate **blocchi**, della stessa dimensione delle pagine.

Idea: salvare le pagine di memoria (virtuale) nei blocchi di memoria (fisica).

Se la dimensione di pagina è 2^k , si usano k bit per indirizzare una locazione all'interno di una pagina (offset).

Nell'esempio consideriamo pagine da 4K locazioni (quindi servono 12 bit per l'offset).

(a) Primi 64KB degli indirizzi virtuali suddivisi in 16 pagine da 4KB ciascuno, assumendo quindi 8 bit di parola. (b) Memoria principale di 32KB suddivisa in 8 blocchi di memoria da 4KB.

Pagina	Indirizzi virtuali
15	61440 – 65535
14	57344 – 61439
13	53248 – 57343
12	49152 – 53247
11	45056 – 49151
10	40960 – 45055
9	36864 – 40959
8	32768 – 36863
7	28672 – 32767
6	24576 – 28671
5	20480 – 24575
4	16384 – 20479
3	12288 – 16383
2	8192 – 12287
1	4096 – 8191
0	0 – 4095

(a)

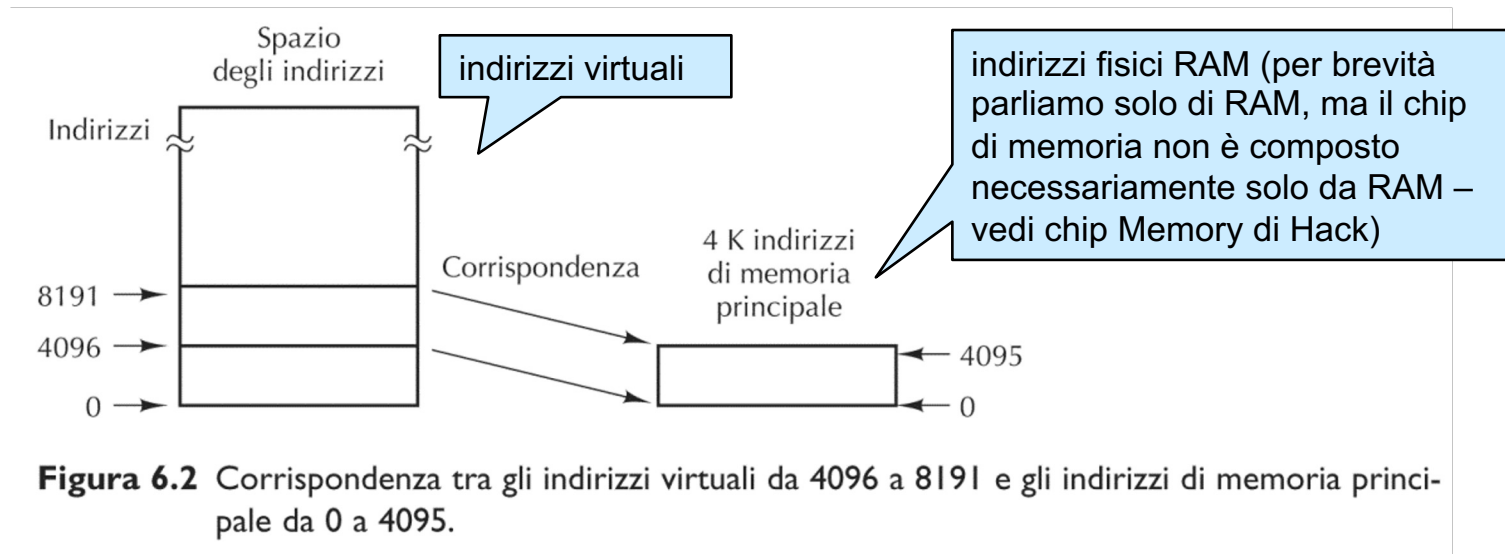
memoria virtuale molto più grande della memoria fisica

Primi 32 K di memoria principale	
Blocco di memoria	Indirizzi fisici
7	28672 – 32767
6	24576 – 28671
5	20480 – 24575
4	16384 – 20479
3	12288 – 16383
2	8192 – 12287
1	4096 – 8191
0	0 – 4095

(b)

Tradurre indirizzi virtuali in indirizzi fisici

Bisogna “**mappare**” (o “risolvere”) gli indirizzi virtuali a cui si accede negli indirizzi fisici dove il dato è effettivamente salvato (assumendo che sia in memoria al momento).



In figura, accedere per esempio all'indirizzo virtuale $4098 = 4096 + \text{offset } 2$ deve determinare un accesso in memoria primaria all'indirizzo fisico $2 = 0 + \text{offset } 2$ (offset=spiazzamento, cf. la modalità di *indirizzamento indicizzato*).

Questo mapping viene fatto dal S.O. in modo **trasparente** al programma.

Come tradurre: la tabella delle pagine

Esiste hardware dedicato (chiamato **Memory Management Unit – MMU**) che si occupa di tradurre un indirizzo virtuale nel corrispondente indirizzo fisico.

Si usa una “**tabella delle pagine**” che indica, per ogni pagina, se questa è al momento in memoria, e in caso positivo indica in quale blocco.

L'indice di pagina si ottiene guardando l'indirizzo virtuale, esclusi i k bit usati per identificare l'offset.

La tabella indica quali bit inserire al posto di questo indice di pagina.

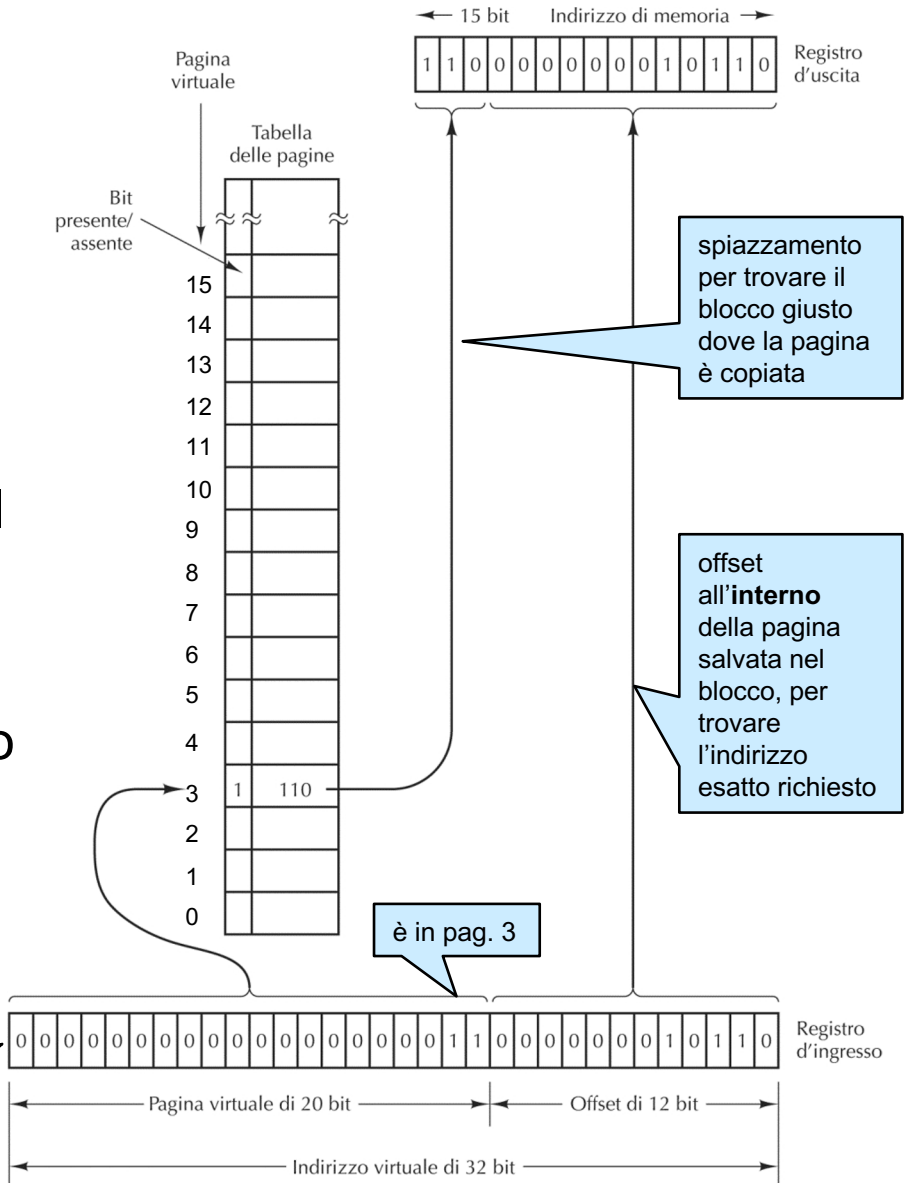
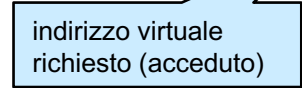


Figura 6.4 Formazione di un indirizzo fisico a partire da un indirizzo virtuale.

Tabella delle pagine

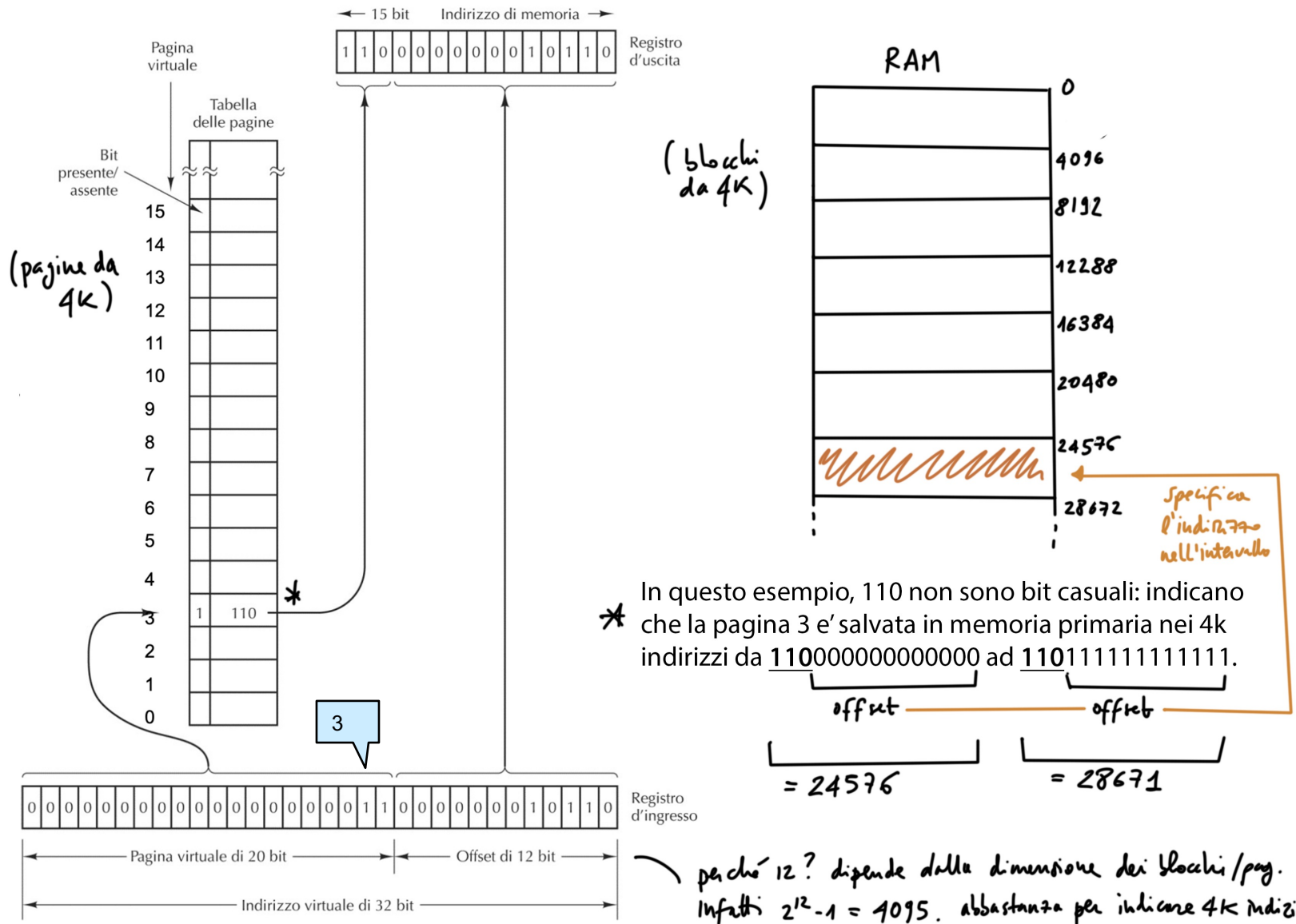


Figura 6.4 Formazione di un indirizzo fisico a partire da un indirizzo virtuale.

Page faults

Come determinare le pagine da tenere in memoria?

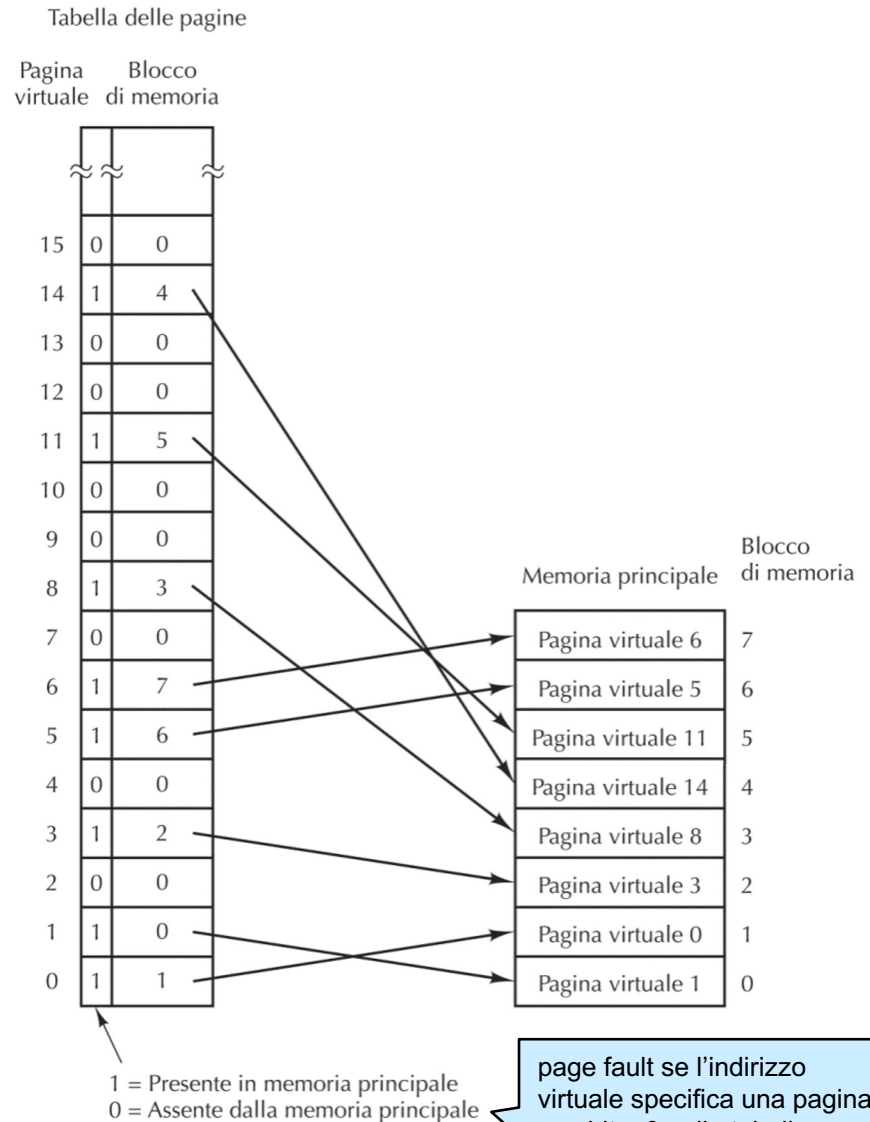
L'insieme delle pagine alle quali un programma deve accedere non è noto all'inizio (in generale).

Quando si accede ad una pagina non al momento in memoria avviene un **trap** detto "**page fault**".

Il gestore di tale trap deve rimuovere dalla memoria una pagina, per far posto per la pagina richiesta (che viene quindi copiata in memoria).

Questa "sostituzione" di pagina in un blocco si chiama **page swapping**.

La scelta di quale blocco liberare (quindi quale pagina rimuovere) viene presa dai cosiddetti *algoritmi di paginazione*.



Algoritmi di paginazione

In generale, possiamo:

- Attendere che il programma faccia richieste di accesso a memoria prima di copiare la pagina richiesta. Le pagine in memoria saranno solo pagine effettivamente necessarie al programma (o un sottoinsieme, se non ci sono abbastanza blocchi).

Oppure:

- Ad una richiesta, utilizzare il principio di località (ne avete parlato nel modulo 1 – vedi) per copiare in memoria anche altre pagine adiacenti. In questo caso potremmo avere in memoria pagine non realmente necessarie. Similmente, si potrebbe tentare di predire le pagine che serviranno, basandosi sul programma.

In quanto segue (e negli esercizi), ci focalizziamo sul primo approccio. Ma algoritmi assolutamente analoghi si possono definire facilmente anche per il secondo approccio.

Algoritmi di paginazione: LRU e FIFO

Due tipici algoritmi di paginazione (~ tipi/famiglie di algoritmi) per determinare quale pagina rimuovere da un blocco quando si verifica un page fault:

- **LRU (Least Recently Used)**

- Si rimuove dalla memoria la pagina che da più tempo non viene acceduta (in lettura/scrittura);
- Può essere implementato tramite una “lista”: quando si accede una pagina la si re-inserisce in coda, e quando serve rimuovere una pagina, si sceglie quella in testa.

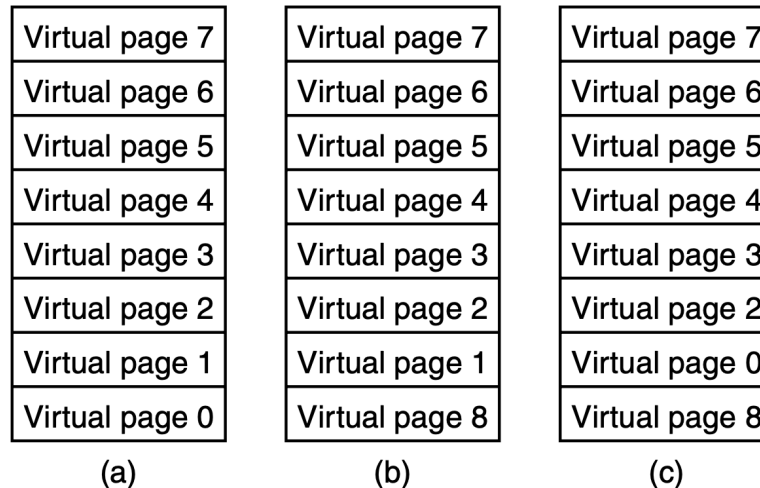
- **FIFO (First-In First-Out)**

- Si rimuove la pagina in memoria da più tempo;
- Può essere implementato da una coda.
Oppure nel seguente modo: ogni blocco ha un contatore, inizializzato a 0 quando il blocco riceve una pagina, ed incrementato ad ogni page fault. Quando serve liberare un blocco si sceglie quello con il contatore più grande (intuizione: si "contano" i page fault a cui la pagina ha sopravvissuto).

Osservazione: caso critico per LRU

Immaginiamo un programma che esegue un ciclo, il cui corpo accede in progressione ad indirizzi virtuali appartenenti a 9 pagine distinte (per semplicità, pagine dalla 0 alla 8, in ordine), ma la memoria fisica ha solo 8 blocchi: ci sarà sempre una pagina necessaria al programma ma che non è in memoria.

Durante l'esecuzione di un ciclo, si arriverà ad un certo punto ad avere le pagine 0-7 in memoria (fig. a – in quale blocco salviamo ogni pagina non è rilevante). Cercando di accedere poi alla pagina 8, un algoritmo LRU rimuove la pagina 0 per far posto (fig. b). Ora il corpo del ciclo termina, ed occorre eseguirlo da capo, quindi la prima pagina ad essere acceduta sarà la pagina 0. Per far posto, LRU rimuove la pagina 1 (fig. c). Ma questa sarà anche la prossima pagina acceduta, etc.



Attenzione

Non confondere la paginazione (*paging*) della memoria virtuale con il caching (in particolare il *page caching*): il caching non ha a che fare con virtualizzazione della memoria (utilizzare un indirizzamento virtuale più vasto), ha come obiettivo le prestazioni, e riguarda l'accesso ai dati da/tra CPU a memoria primaria.

Dirty bit

Quando si rende necessario liberare un blocco (page fault), se la pagina non è stata modificata non serve **ricopiarla** in memoria secondaria!

Per tenere traccia di questo, si può usare la tecnica del “**dirty bit**”:

- Ogni blocco in memoria primaria ha un suo bit associato
- Quando si carica una nuova pagina nel blocco, si pone a 0 il bit corrispondente
- Quando la pagina in un blocco viene acceduta in scrittura, il bit viene posto a 1
- Quando si deve liberare il blocco, si salva la corrispondente pagina in memoria secondaria solo se il bit è a 1.

Esercizio

Si consideri un sistema di virtualizzazione della memoria basato su paginazione con algoritmo **LRU** e dirty bit.

Si assuma che inizialmente sia caricata solo pagina 1 (su cui sono stati fatti accessi in scrittura), e che la memoria principale sia di 3 blocchi.

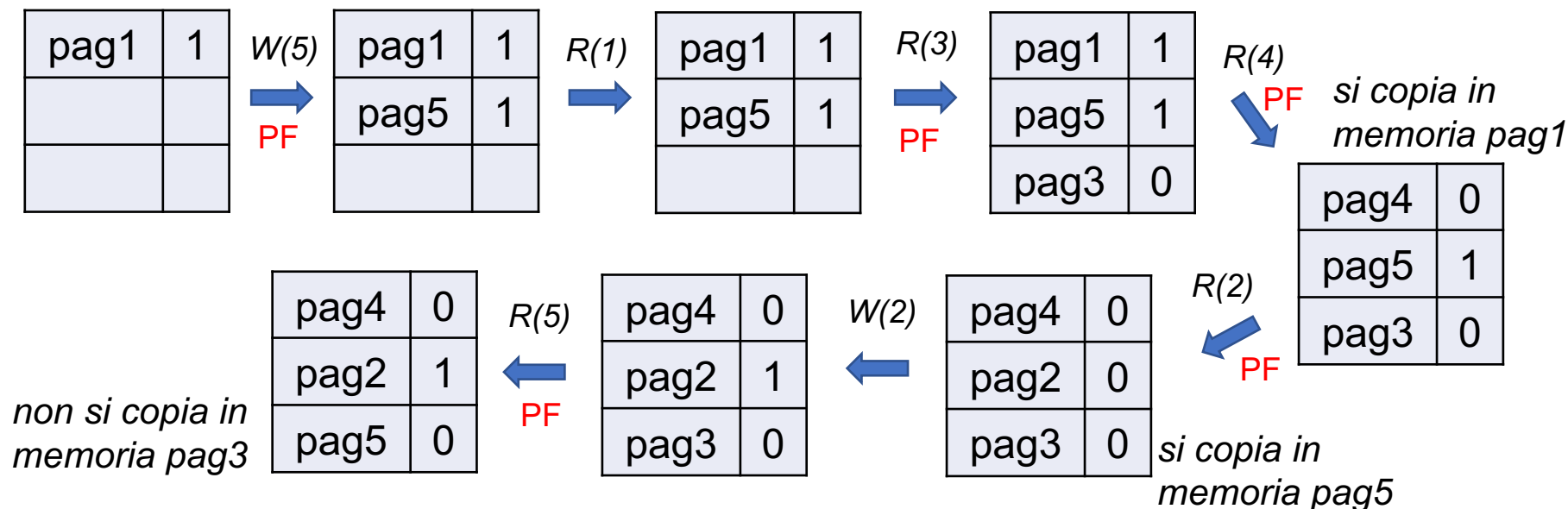
Illustrare cosa succede (a parole e diagramma) se vengono fatti i seguenti accessi: scrittura pagina 5, lettura pagina 1, lettura pagina 3, lettura pagina 4, lettura pagina 2, scrittura pagina 2, lettura pagina 5. Utilizzare la terminologia appropriata.

Esercizio: soluzione

Si consideri un sistema di virtualizzazione della memoria basato su paginazione con algoritmo **LRU** e dirty bit.

Si assuma che inizialmente sia caricata solo pagina 1 (su cui sono stati fatti accessi in scrittura), e che la memoria principale sia di 3 blocchi.

Illustrare cosa succede (a parole e diagramma) se vengono fatti i seguenti accessi: scrittura pagina 5, lettura pagina 1, lettura pagina 3, lettura pagina 4, lettura pagina 2, scrittura pagina 2, lettura pagina 5.



Esercizio

Si consideri un sistema di virtualizzazione della memoria basato su paginazione con algoritmo **FIFO** e dirty bit.

Si assuma che inizialmente sia caricata solo pagina 3 (su cui sono stati già fatti accessi in scrittura), e che la memoria principale sia di 3 blocchi.

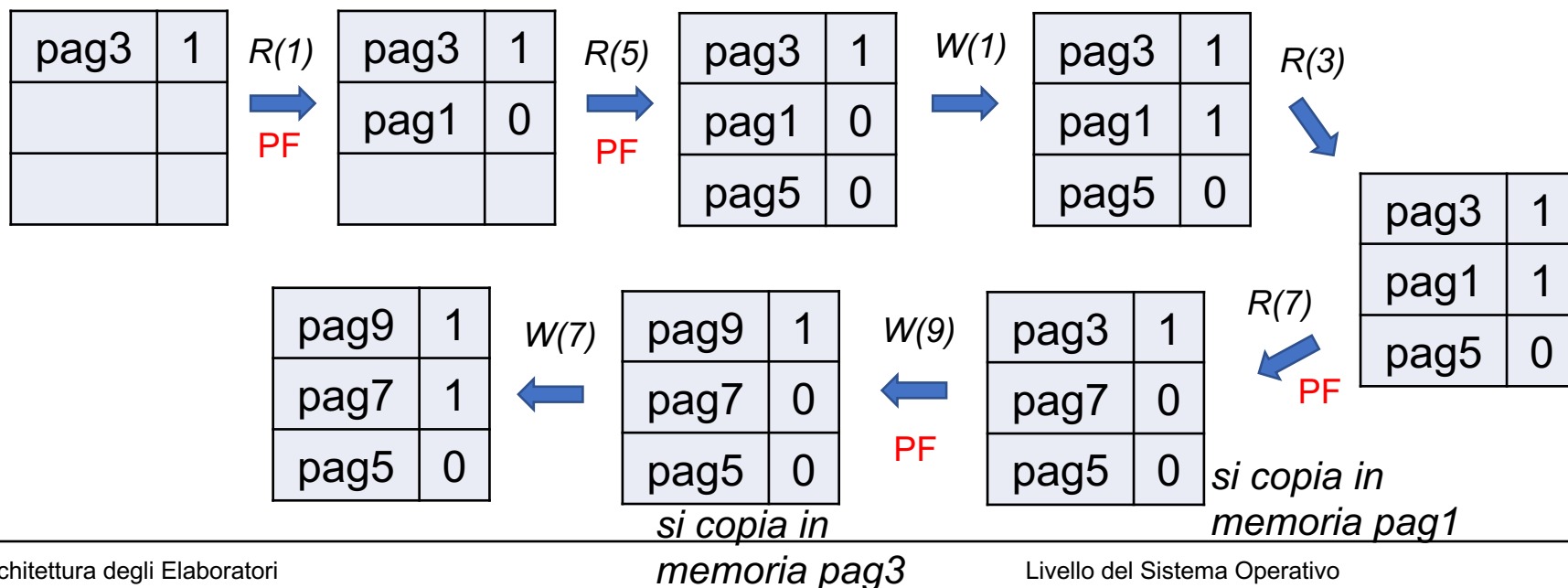
Si descriva (a parole e diagramma) cosa succede se vengono fatti i seguenti accessi: lettura pagina 1, lettura pagina 5, scrittura pagina 1, lettura pagina 3, lettura pagina 7, scrittura pagina 9, scrittura pagina 7. Utilizzare la terminologia appropriata.

Esercizio: soluzione

Si consideri un sistema di virtualizzazione della memoria basato su paginazione con algoritmo **FIFO** e dirty bit.

Si assuma che inizialmente sia caricata solo pagina 3 (su cui sono stati già fatti accessi in scrittura), e che la memoria principale sia di 3 blocchi.

Si descriva (a parole e diagramma) cosa succede se vengono fatti i seguenti accessi: lettura pagina 1, lettura pagina 5, scrittura pagina 1, lettura pagina 3, lettura pagina 7, scrittura pagina 9, scrittura pagina 7.



Memoria virtuale: segmentazione

Frammentazione interna

Supponiamo che ad un programma serva una quantità di memoria che non sia un multiplo della dimensione delle pagine: l'ultima pagina non verrà completamente utilizzata.

Esempio:

- Consideriamo un programma che richiede 26.000 byte
- In esecuzione su una macchina con pagine da 4KB (4096 byte)
- Il programma userà in modo completo 6 pagine ($6 \times 4.096 = 24.576$)
- Nell'ultima pagina, la settima, si useranno solamente $26.000 - 24.576 = 1.424$ byte
- I restanti $4.096 - 1.424 = 2.672$ byte non verranno utilizzati

Questo fenomeno viene chiamato **frammentazione interna**.

Esiste anche la frammentazione esterna (che richiede però di introdurre prima la nozione di segmento).

Segmentazione: memoria segmentata

Finora abbiamo pensato a una memoria (virtuale o reale) di k locazioni come a un unico spazio di indirizzamento con indirizzi da 0 a $k-1$.

Tuttavia una memoria è comunemente organizzata logicamente in porzioni distinte utilizzate con distinti scopi/modalità, diritti di accesso, protezioni in scrittura, etc. Ciascun segmento ha il proprio spazio di indirizzamento.

E' simile ad avere disponibili contemporaneamente diverse memorie. Esempio relativo a un compilatore:

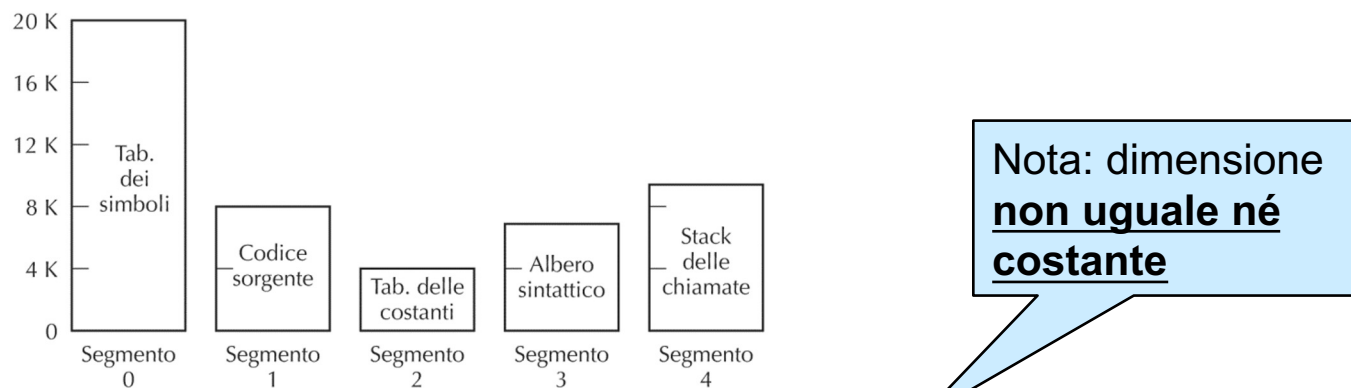


Figura 6.8 Una memoria segmentata consente a ciascuna tabella di crescere o decrescere indipendentemente dalle altre.

Segmentazione: memoria segmentata

I segmenti sono visibili anche al programmatore e al codice assembly.

Possiamo quindi in assembly scrivere/leggere ad un certo indirizzo di un certo segmento, usare indirizzamento indicizzato (cioè con offset) nel segmento, etc. Segmenti tipici usati in assembly: code segment (come la ROM Hack), data segment, stack segment, extra segment.

A differenza delle paginazione, **la segmentazione non è trasparente al programma(tore)!**

Segmentazione (continua)

- Ogni segmento di dimensione n , usa gli indirizzi da 0 a $n-1$
- Per disambiguare l'uso di indirizzi presenti in più segmenti (ad esempio l'indirizzo 0 è presente in tutti i segmenti!) gli indirizzi dovranno essere arricchiti con l'indicazione del relativo segmento
- L'indirizzo **(n,k)** indica la locazione di indirizzo **k** del segmento **n**

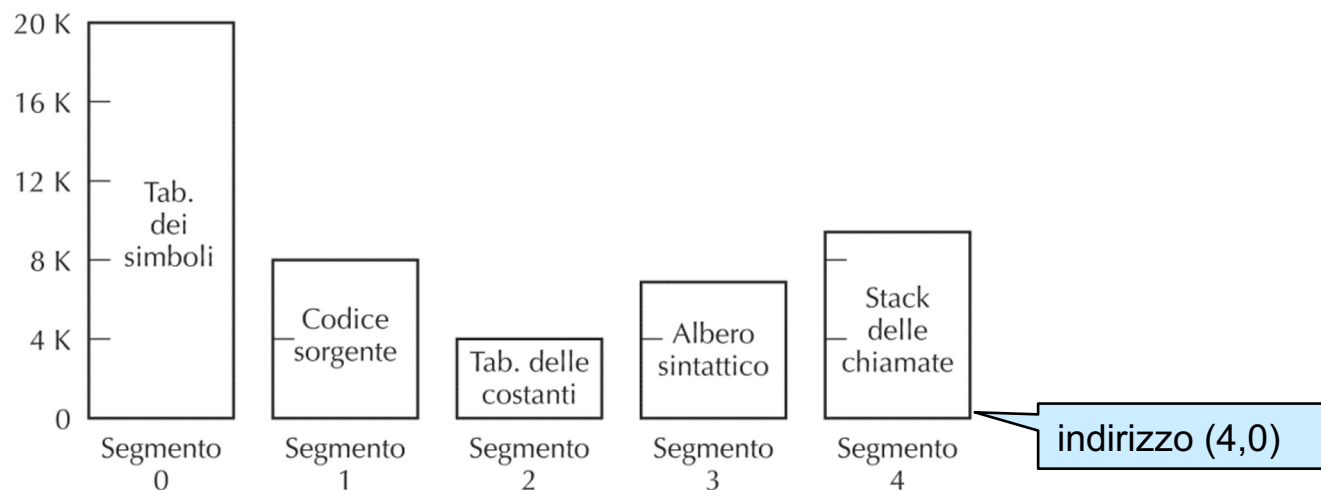


Figura 6.8 Una memoria segmentata consente a ciascuna tabella di crescere o decrescere indipendentemente dalle altre.

Implementazione della segmentazione

Come nella paginazione, possiamo tenere in memoria solo alcuni segmenti al momento utilizzati, lasciando in memoria secondaria i segmenti non necessari.

- Quando serve inserire un nuovo segmento se ne toglie un altro, tramite *swapping* di segmenti.
- A seguito di swapping si possono generare zone non utilizzate dette **lacune**. Questo fenomeno viene detto “**frammentazione esterna**”, da risolvere tramite **compattamento**.

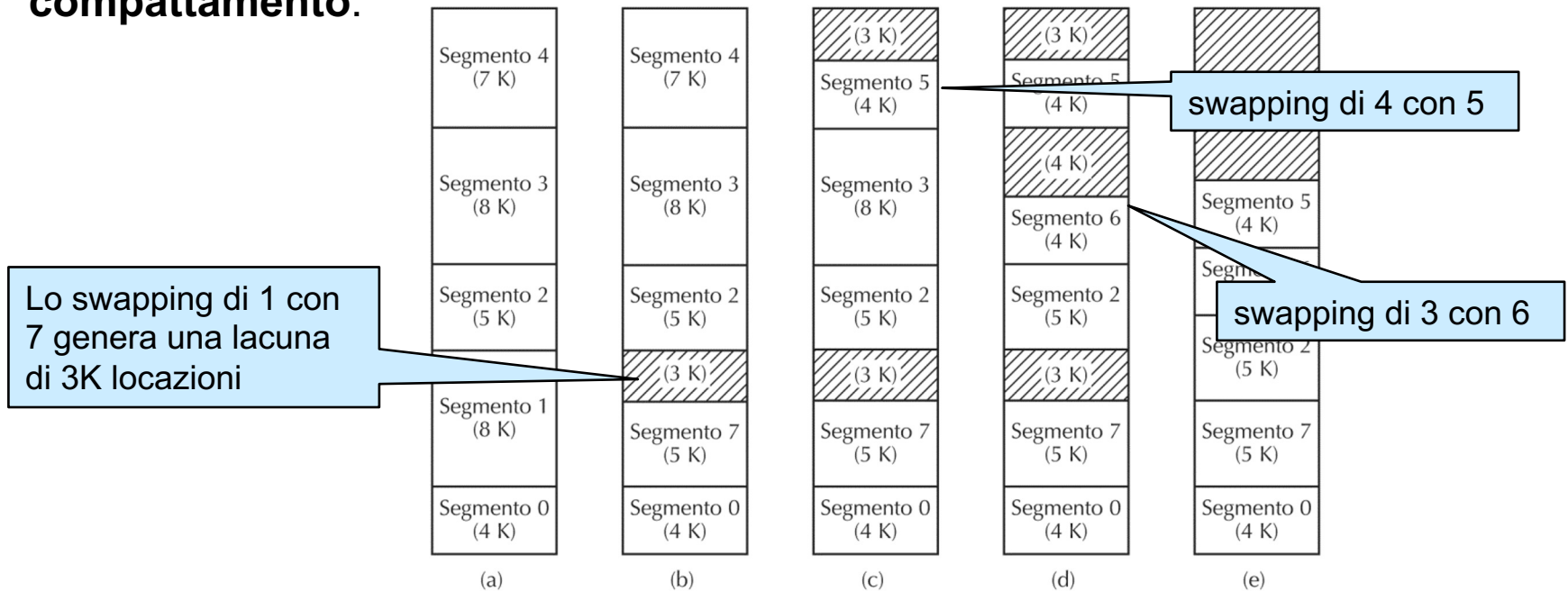


Figura 6.10 (a)-(d) Incremento della frammentazione esterna. (e) Rimozione della frammentazione esterna mediante compattamento.

Gestione della frammentazione esterna

Si sono ideate varie tecniche di *swapping* di segmenti per cercare di ridurre la frammentazione esterna. Tra le tecniche più usate menzioniamo:

- **Best fit:** si inserisce il nuovo segmento nella lacuna più piccola sufficiente per il segmento da inserire;
- **First fit:** si scorrono le lacune circolarmente selezionando la prima sufficientemente grande per il segmento da inserire

Nel caso medio, First fit è più veloce nella scelta, ed inoltre limita la creazione di piccole e inutili lacune.

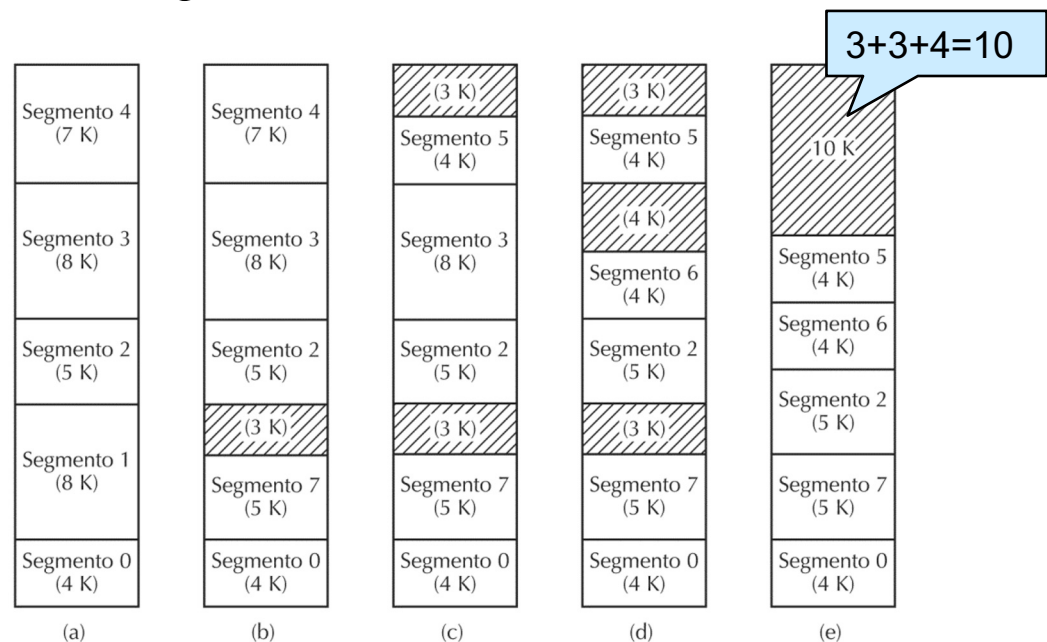


Figura 6.10 (a)-(d) Incremento della frammentazione esterna. (e) Rimozione della frammentazione esterna mediante compattamento.

Memoria virtuale: paginazione + segmentazione

Combinare segmentazione e paginazione

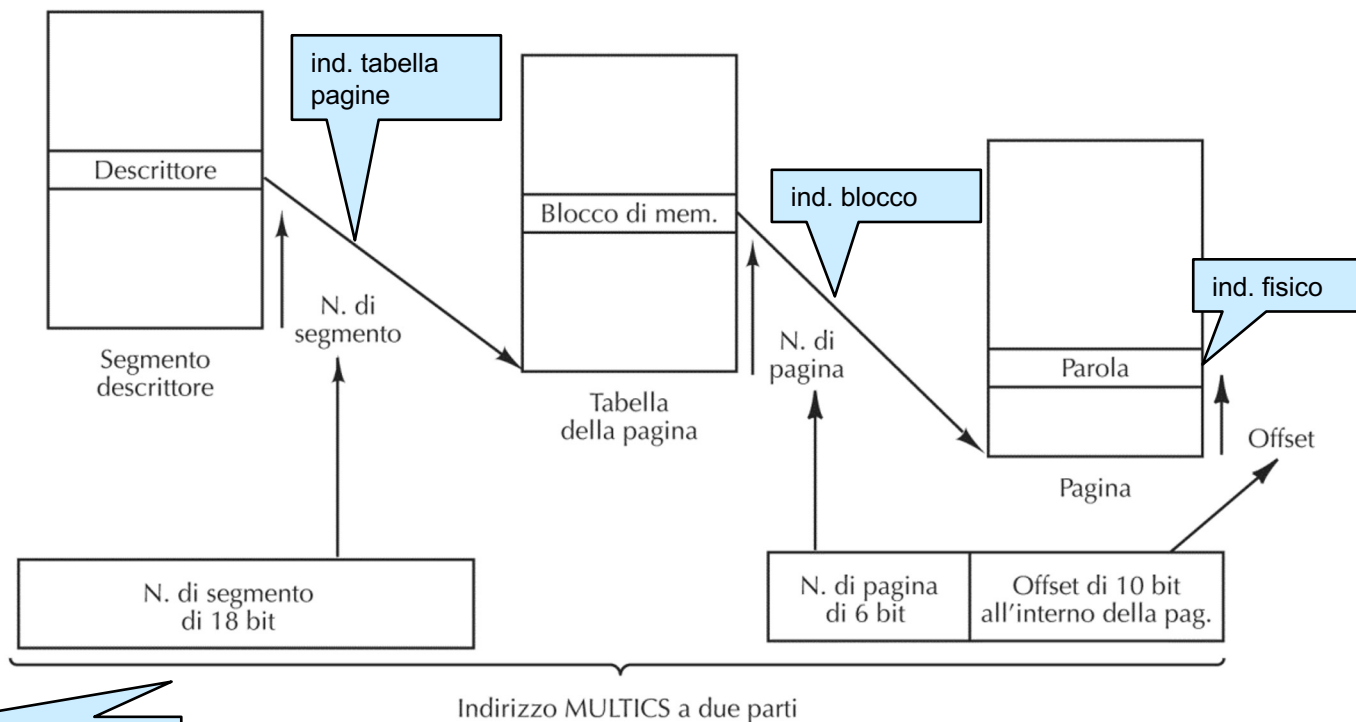
Un modo per evitare la frammentazione esterna, oltre al compattamento, è combinare segmentazione e paginazione.

Intuizione:

- La memoria virtuale è paginata e segmentata. Quindi è divisa logicamente in segmenti, ed ogni segmento è diviso logicamente in pagine.
- Solo le pagine necessarie (accedute dal programma) vengono copiate in memoria principale esattamente come visto prima (cf. page swapping)
- Quindi si riduce la porzione di ciascun segmento che effettivamente si tiene in memoria, ma soprattutto visto che le pagine sono sempre di dimensione uguale ai blocchi, si evita la frammentazione esterna: si copiano in memoria sempre multipli della dimensione dei blocchi
- C'è però più lavoro per la MMU per tradurre gli indirizzi, che deve **utilizzare 2 tabelle**: una per i segmenti (a sua volta salvata in un segmento) per reperire la tabella delle pagine del segmento richiesto:
 - 1 tabella dei segmenti
 - 1 tabella delle pagine per ogni segmento

Combinare segmentazione e paginazione

- 1 tabella dei segmenti
- 1 tabella delle pagine **per ogni segmento**



Conversione di un indirizzo MULTICS a due parti in un indirizzo fisico.

ora un indirizzo virtuale deve specificare:

- segmento
- pagina
- offset di pagina

Linking e collegamento dinamico
per la compilazione (vedi anche le slide su C)

Come viene preparato un programma per essere eseguito

- Un programma può essere composto da unità distinte
- Ognuna di tali procedure può essere compilata separatamente per generare i cosiddetti “moduli oggetto” (vedi slide su C)
- Per poter avere un programma pronto per l'esecuzione si rende necessaria una fase detta di “collegamento” fatta dal linker.

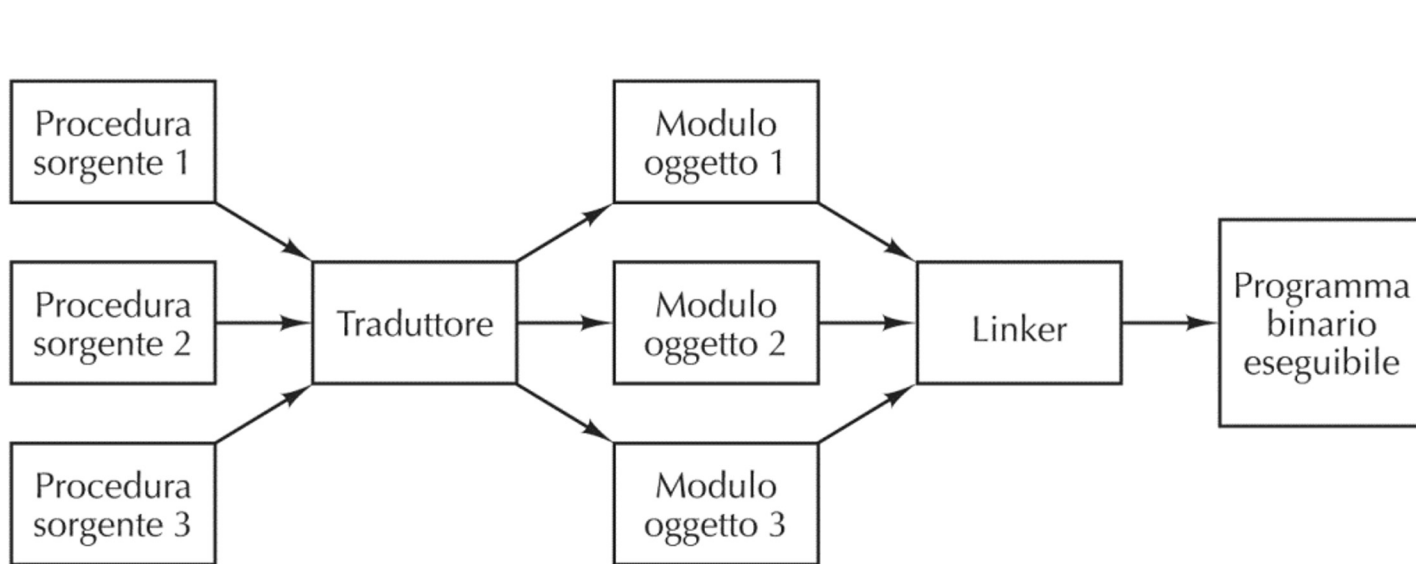


Figura 7.12 Generazione tramite un linker di un programma eseguibile binario a partire da un gruppo di file sorgente tradotti indipendentemente.

Come funziona il linker

Ogni modulo oggetto (come i file .o che abbiamo visto) viene realizzato assumendo di usare un proprio spazio di indirizzi che inizia da 0. Consideriamo, per esempio, 4 moduli oggetto da collegare (in figura si assume un generico linguaggio assembly).

MOVE: istruzione assembly di copia dati.
Per esempio: da registri a memoria

CALL: istruzione assembly per chiamata di procedura (la procedura potrebbe essere definita in altro modulo). In figura: CALL D significa eseguire la «funzione D» implementata dall'intero modulo D. Anche se di solito un modulo esporta (agli altri) più di una funzione.

BRANCH: istruzione assembly di salto

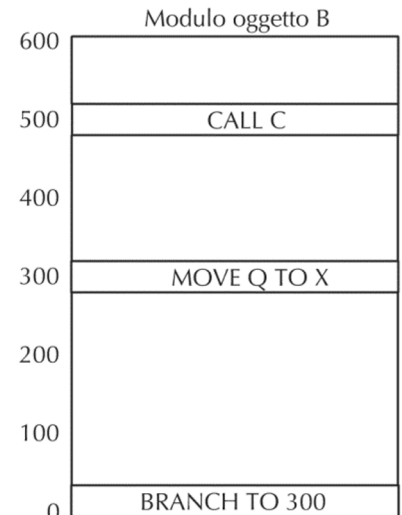
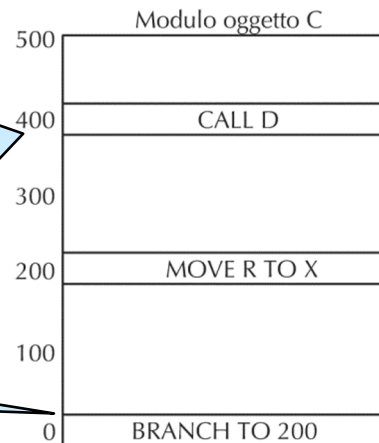
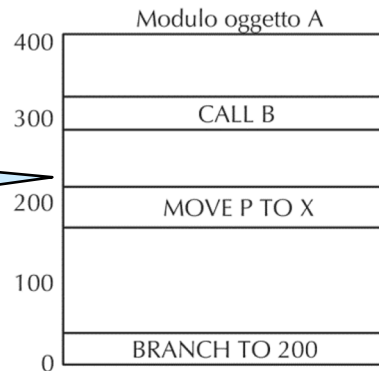


Figura 7.13 Ogni modulo ha il proprio spazio d'indirizzi che inizia da 0.

Problema!

Gli indirizzi usati in (come operandi di) queste istruzioni, fanno riferimento allo spazio di indirizzamento del proprio modulo: dobbiamo convertire gli indirizzi in indirizzi globali (indirizzi di memoria).

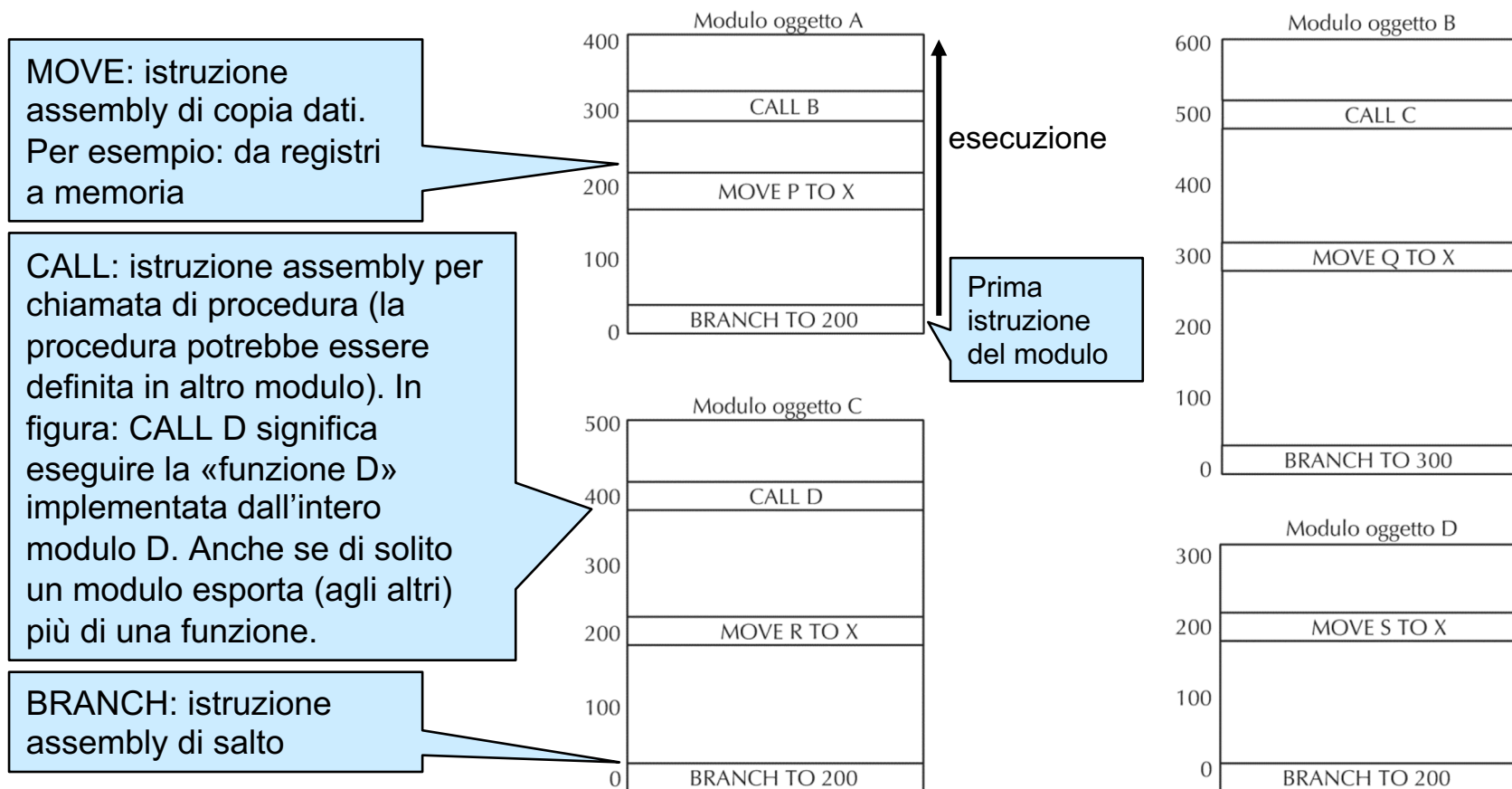


Figura 7.13 Ogni modulo ha il proprio spazio d'indirizzi che inizia da 0.

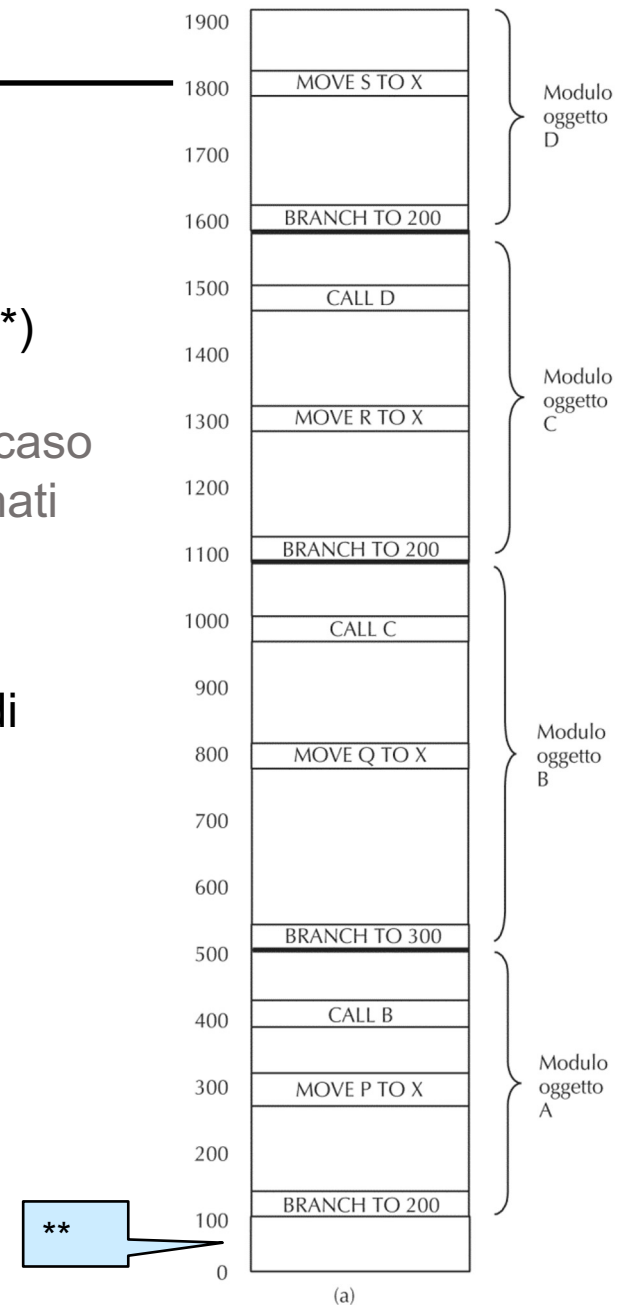
Collegamento

Il linker decide l'ordine in cui collegare i moduli.
Inizialmente, si dispongono sequenzialmente.

All'inizio inserisce informazioni di servizio necessarie (**) per eseguire il programma (ad esempio il vettore di interrupt, utilizzato per identificare l'ISR da eseguire in caso di interrupt – vedi slide: non tutti gli interrupt sono originati da dispositivi).

Per ogni modulo collegato si tiene traccia della sua lunghezza (l) e dell'indirizzo di inizio nel nuovo spazio di indirizzamento (i):

- Modulo A l:400 , i:100
- Modulo B l:600 , i:500
- Modulo C l:500 , i:1100
- Modulo D l:300 , i:1600



Rilocazione

Si rende necessario modificare gli indirizzi presenti all'interno dei moduli oggetto: **rilocazione**.

Parte facile: sistemare riferimenti **interni** (cioè al medesimo modulo) incrementandolo del valore dell'indirizzo di inizio del modulo.

Occorre tener traccia di quali istruzioni contengono riferimenti e quindi hanno bisogno di rilocazione.

Si usa il *dizionario di rilocazione*.

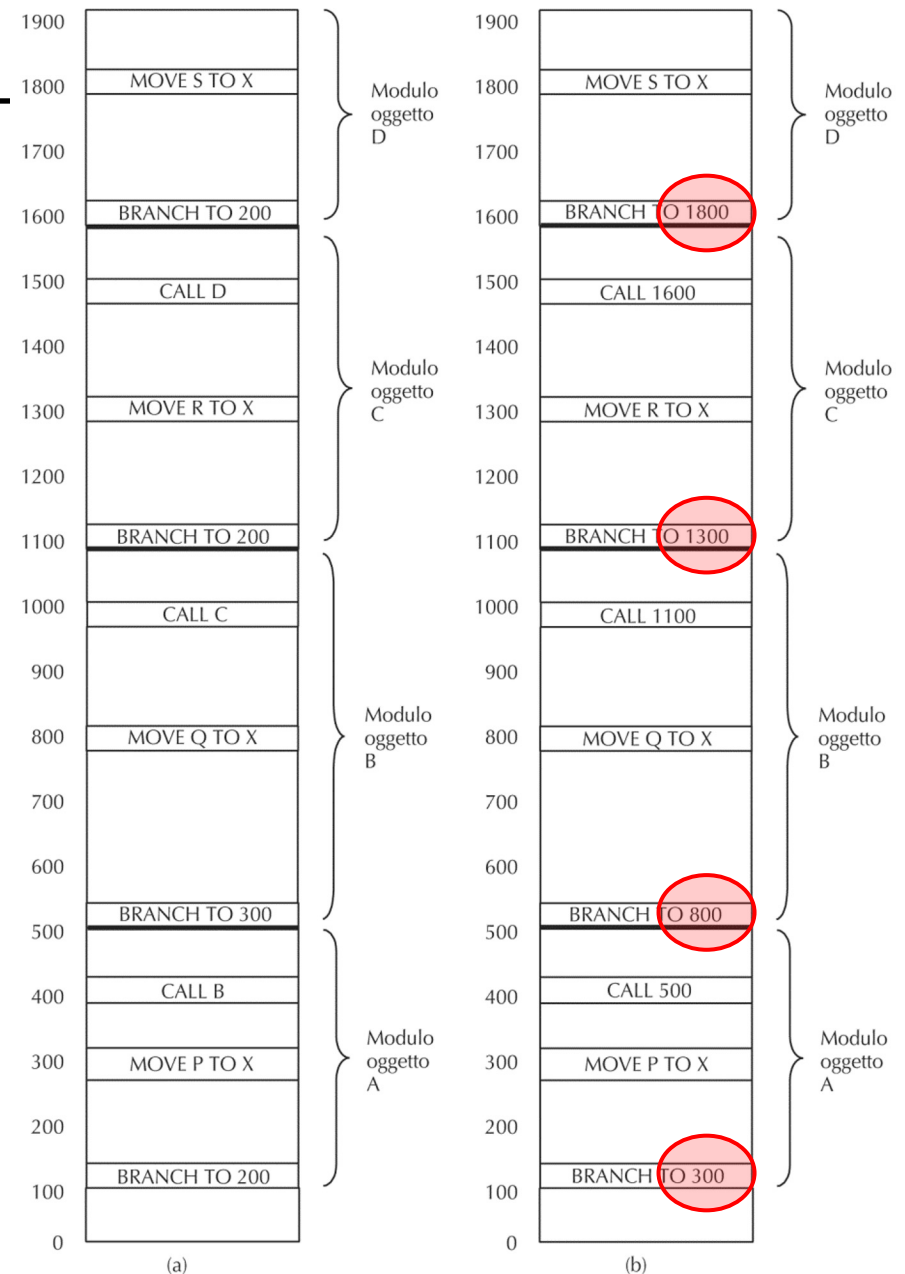


Figura 7.14 (a) I moduli oggetto della Figura 7.13 dopo essere stati posizionati nell'immagine binaria, ma prima di essere rilocati e collegati. (b) Gli stessi moduli oggetto dopo aver effettuato il collegamento e la rilocazione.

Linker: riferimenti esterni

Per gestire i riferimenti esterni servono più informazioni

Esempio: il modulo C deve dire a che indirizzo interno corrisponde il simbolo "C" che "esporta", cioè rende invocabile da altri moduli.

Ed il modulo B deve indicare al linker il fatto che ha bisogno del simbolo esterno "C" esportato da C.

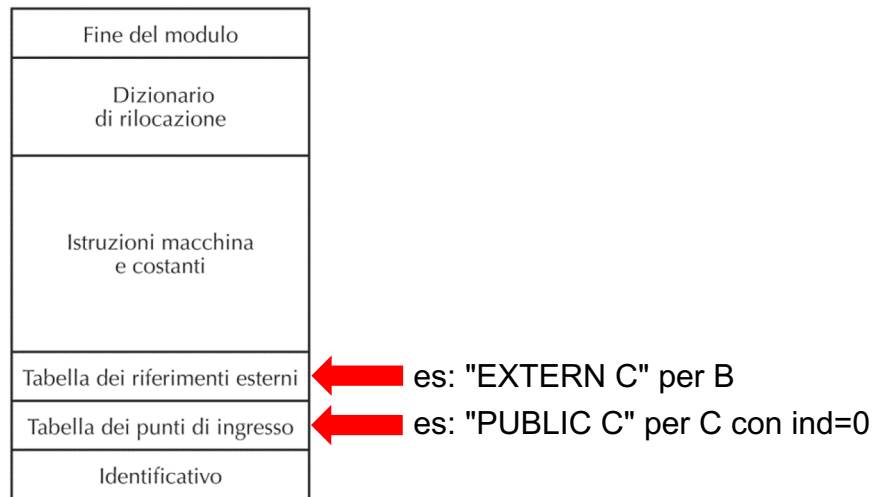


Figura 7.15 Struttura interna di un modulo oggetto prodotto da un traduttore. — Il campo *identificativo* è il primo.

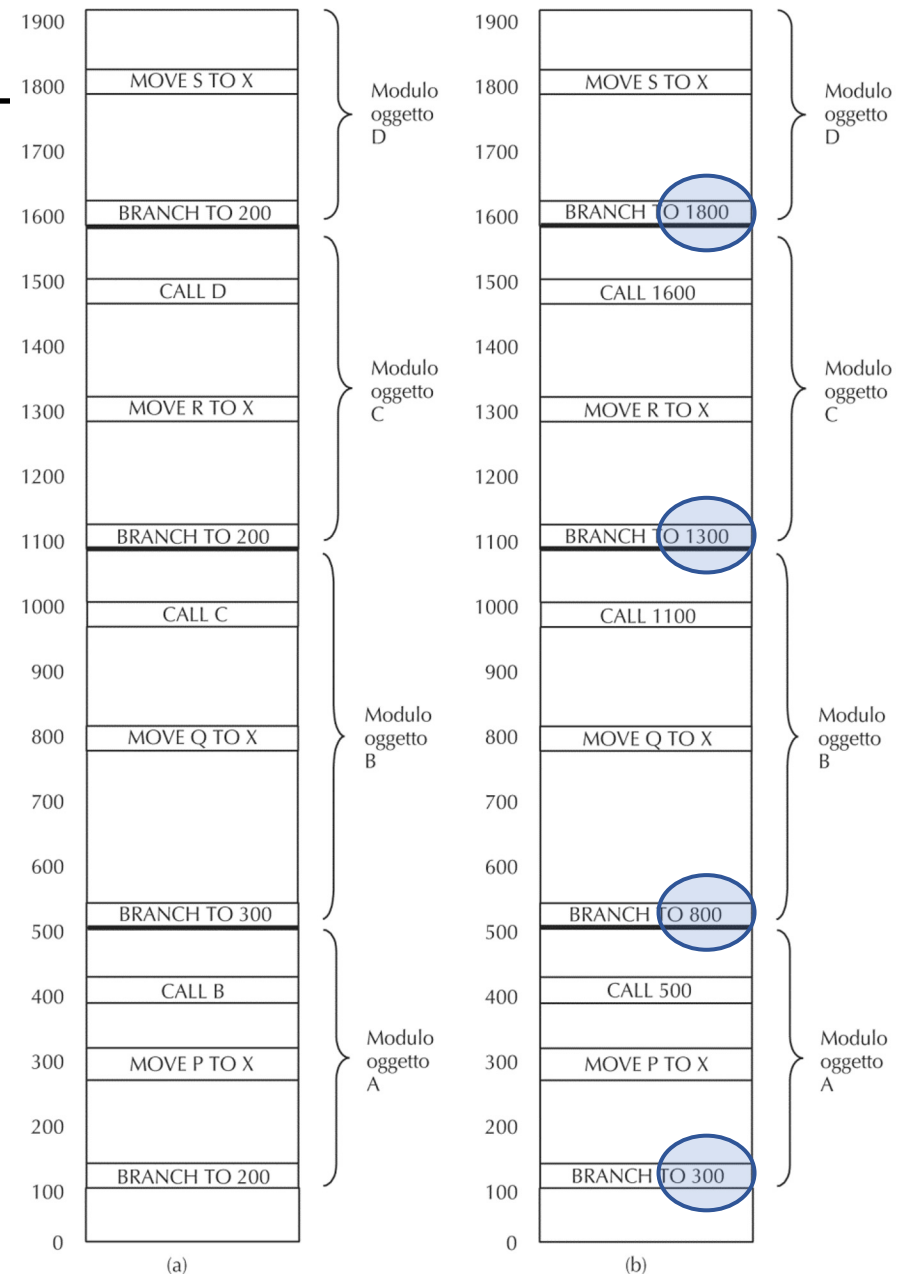


Figura 7.14 (a) I moduli oggetto della Figura 7.13 dopo essere stati posizionati nell'immagine binaria, ma prima di essere rilocati e collegati. (b) Gli stessi moduli oggetto dopo aver effettuato il collegamento e la rilocazione.

Linker: riferimenti esterni

Un codice assembly che abbiamo già incontrato... (torre di Hanoi).

Questo ovviamente è il codice, e non il modulo oggetto come nella slide precedente, ma possiamo vedere come le informazioni necessarie siano tutte qui.

Per esempio, si vuole esportare la funzione towers, con punto di ingresso l'indirizzo locale corrispondente all'etichetta towers.

Si dichiara anche di aver bisogno di una funzione esterna printf (quella di C: da assembly si può eseguire codice C –compilato, ovviamente– e vice-versa).

```
.686
.MODEL FLAT
PUBLIC _towers
EXTERN _printf:NEAR
.CODE
towers: PUSH EBP
        MOV EBP, ESP
        CMP [EBP+8], 1
        JNE L1
        MOV EAX, [EBP+16]
        PUSH EAX
        MOV EAX, [EBP+12]
        PUSH EAX
        PUSH OFFSET FLAT:format
        CALL _printf
        ADD ESP, 12
        JMP Done
L1:      MOV EAX, 6
        SUB EAX, [EBP+12]
        SUB EAX, [EBP+16]
        MOV [EBP+20], EAX
        PUSH EAX
        MOV EAX, [EBP+12]
        PUSH EAX
        MOV EAX, [EBP+8]
        DEC EAX
        PUSH EAX
        CALL _towers
        ADD ESP, 12
        MOV EAX, [EBP+16]
        PUSH EAX
        MOV EAX, [EBP+12]
        PUSH EAX
        PUSH 1
        CALL _towers
        ADD ESP, 12
        MOV EAX, [EBP+12]
        PUSH EAX
        MOV EAX, [EBP+20]
        PUSH EAX
        MOV EAX, [EBP+8]
        DEC EAX
        PUSH EAX
        CALL _towers
        ADD ESP, 12
Done:    LEAVE
        RET 0

.DATA
format DB "Move disk from %d to %d\n" ; format string
END
```

; compile for Core i7 class processor

; export 'towers'

; import printf

; save EBP (frame pointer) and decrement ESP

; set new frame pointer above ESP

; if (n == 1)

; branch if n is not 1

; printf(" ...", i, j);

; note that parameters i, j and the format

; string are pushed onto the stack

; in reverse order. This is the C calling convention

; offset flat means the address of format

; call printf

; remove params from the stack

; we are finished

; start k = 6 - i - j

; EAX = 6 - i

; EAX = 6 - i - j

; k = EAX

; start towers(n - 1, i, k)

; EAX = i

; push i

; EAX = n

; EAX = n - 1

; push n - 1

; call towers(n - 1, i, 6 - i - j)

; remove params from the stack

; start towers(1, i, j)

; push j

; EAX = i

; push i

; push 1

; call towers(1, i, j)

; remove params from the stack

; start towers(n - 1, 6 - i - j, i)

; push i

; EAX = k

; push k

; EAX = n

; EAX = n - 1

; push n - 1

; call towers(n - 1, 6 - i - j, i)

; adjust stack pointer

; prepare to exit

; return to the called

Linker: riferimenti esterni (soluzioni)

Grazie a queste informazioni, più la tabella con gli indirizzi di inizio dei vari moduli discussa in precedenza, il linker riesce a rilocare correttamente anche i riferimenti **esterni**.

Se il primo modulo non è caricato in memoria a partire da 0 serve una ulteriore rilocazione.

Fine del modulo
Dizionario di rilocazione
Istruzioni macchina e costanti
Tabella dei riferimenti esterni
Tabella dei punti di ingresso
Identificativo

Figura 7.15 Struttura interna di un modulo oggetto prodotto da un traduttore. — Il campo *identificativo* è il primo.

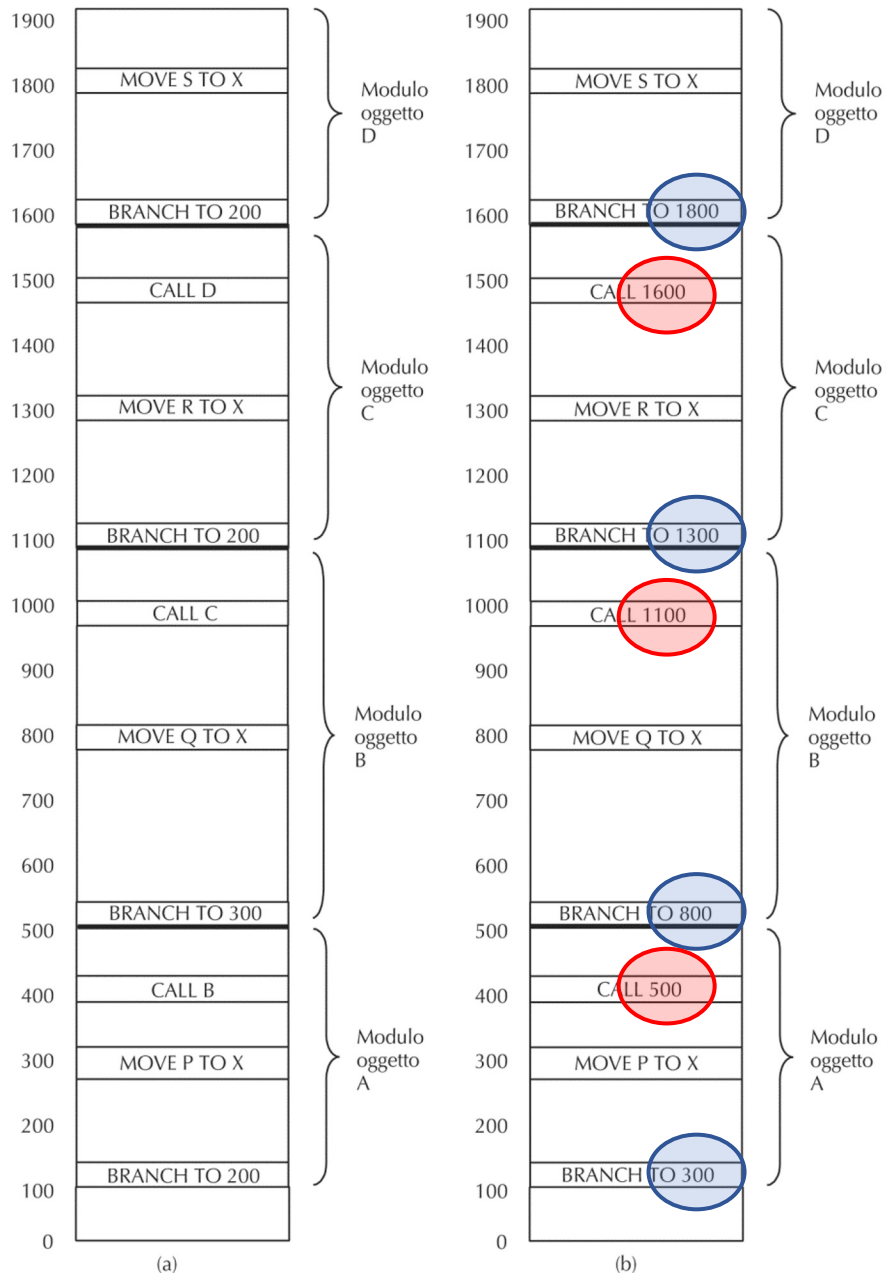


Figura 7.14 (a) I moduli oggetto della Figura 7.13 dopo essere stati posizionati nell'immagine binaria, ma prima di essere rilocati e collegati. (b) Gli stessi moduli oggetto dopo aver effettuato il collegamento e la rilocazione.

Collegamento dinamico

In alcuni casi, le procedure che vengono usate, non vengono inserite nell'eseguibile: vengono collegate **dinamicamente** al momento dell'esecuzione del programma stesso.

Questo può essere utile per procedure che vengono invocate solo in casi rari, o che è lecito assumere siano disponibili/installate nel sistema. Per esempio una libreria legata alla scheda grafica.

Un modo per implementare il collegamento dinamico è il seguente:

- Si compila la chiamata alla procedura “dinamica” tramite una chiamata ad un indirizzo volutamente “sbagliato”, per generare un **trap**;
- Nel programma compilato si inserisce il nome della procedura che si desidera invocare tramite link dinamico;
- La prima volta che si arriva alla chiamata “sbagliata”, il gestore del trap cerca tramite il nome appropriatamente inserito nel codice, l'indirizzo in cui si trova la procedura, e sostituisce nel codice il salto a quell'indirizzo

Vedi prossima slide..

Collegamento dinamico

Se si arriva alla prima istruzione di tipo “indirizzo non valido” si genera un trap.

Il gestore:

- legge la stringa “EARTH” e cerca l’attuale posizione della procedura “EARTH”
- Sostituisce “indirizzo non valido” con l’indirizzo valido di “EARTH” ed esegue la call

In questo modo, future call non genereranno più trap

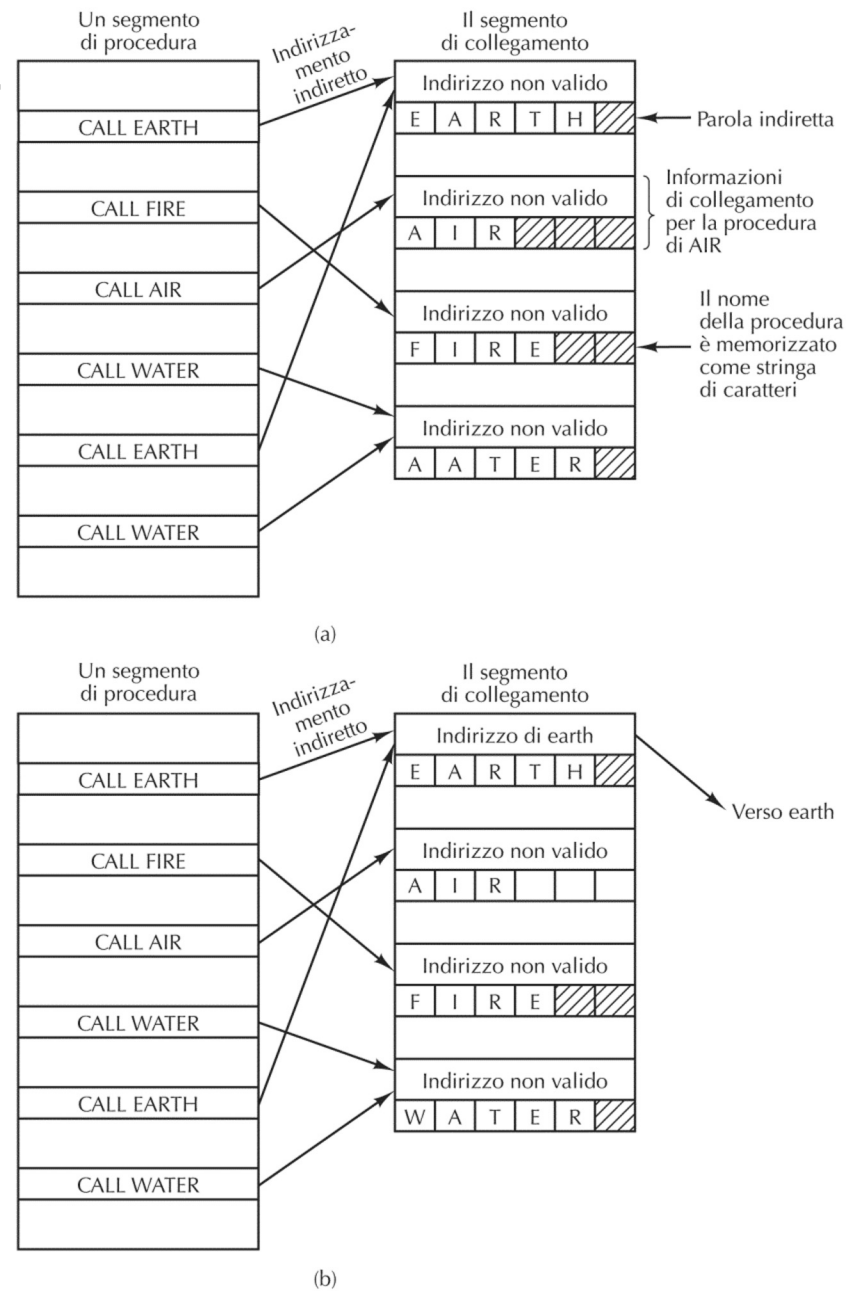


Figura 7.17 Collegamento dinamico. (a) Prima che *EARTH* sia chiamata. (b) Dopo che *EARTH* è stata chiamata e collegata.

Collegamento dinamico (conclusione)

Il collegamento dinamico viene usato per tutta una serie di procedure che solitamente fanno parte delle cosiddette “librerie dinamiche”.

Ad esempio, i sistemi operativi Windows usa le cosiddette **DLL (Dynamic Link Library)** per procedure che possono essere usate da tanti programmi diversi (ad esempio i driver per accedere ai dispositivi di I/O).

Sarebbe infatti inutile inserirle nell'eseguibile di tutti i programmi che le usano, ma si assume siano installate una volta per tutte nel sistema.

Inoltre, in caso di modifica/aggiornamento di tali librerie, sarebbe necessario andare a modificare tutti gli eseguibili in cui queste sono già state inserite.

Nota: Quanto visto su linking e rilocazione non ha a che vedere con la virtualizzazione della memoria. Sarebbe uguale anche con uso di sola memoria fisica e indirizzi fisici.